

PORTADA

MIRAR SI PUEDO PONER COLOR SOLO ESTA PAGINA

ÍNDICE

Letra Arial 12, interlineado de 1.5, márgenes: Izq. 3 cm y Der. 2.5 cm. • Resumen ("abstract") en castellano + inglés o valenciano + inglés. • Formato adecuado de encabezados (título y autor) y pies de página (numeración), exceptuado el índice y la portada que no están numerados. • Imágenes y tablas numeradas y con títulos. Capturas de pantalla de la aplicación comentadas, NO dejadas caer en el documento sin más. 2º DAM • Enlace al prototipo y resultado final de la interfaz. • Redacción con cohesión, coherencia y corrección. • Se debe cuidar la ortografía y expresión gramatical, ilustrando el funcionamiento de la aplicación con capturas de pantalla y esquemas. El texto debe estar cuidadosamente justificado, y el código debidamente indentado. • Se puede incluir código interesante o desafiante al documento, el resto deberá subirse en su totalidad a la actividad de AULES, junto al documento de la memoria. Alternativamente se podrá enviar un documento anexo que contenga enlaces al proyecto en GitHub o enlace de wetransfer para descargar el proyecto. El código deberá estar comentado en inglés.

poner en que lenguaje se desarrollo cada parte
del trabajo muy imporatnteeeeeeeeeeeeee

poner índice (ALINEAR TODO EL DOC) **CAMBIAR FUENTE A MATERIAL 3**

1

Introducción (hacer que se ordenen el orden del proyecto y corrija faltas, poner foto del logo de la app y del instituto, mi nombre y el del profe de proyecto)

LocalHero es mi proyecto de final de curso para proyecto intermodular de 2 DAM 2025/2026.

Mi proyecto llamado **LocalHero** trata sobre una aplicación móvil y una página web en la cual trata sobre un **portal de establecimientos** en el cual, los dueños puedan crear un perfil en ella para poder subir **productos que vayan a desechar** por que se aproxima la fecha de caducidad, renovación de stock o simplemente quieren **donar productos** y que la gente puede ir a por ellos a recogerlos para darles una nueva vida.(extender)



Justificación del proyecto

He elegido este tema de proyecto porque veo que **hay mucho desperdicio** en la actualidad de cosas que se pueden aún usar o darles una **segunda vida**, como por ejemplo en supermercados, panaderías, tiendas de ropa y todo tipo de comercios.

Con este proyecto pretendo que se dé una segunda oportunidad a esos **productos** que iban a ser **desechados** permitiendo a las tiendas que publiquen esos productos y que los usuarios los usen.

Usuarios a los que está dirigida la aplicación

Esta aplicación va dirigida principalmente a **personas** las cuales **quieran reducir el desperdicio** y ayudar al **cuidado del medio ambiente** o simplemente a gente que lo necesite ahorrar dinero y le vaya a dar un buen uso y así evitar la producción masiva de productos.

Además también está dirigido a los **comercios que quieren reducir las pérdidas** de los productos y tener una imagen más sostenible.



Organización y arquitectura del proyecto

La aplicación que he desarrollado sigue una **arquitectura cliente-servidor**, en la cual la información se encuentra en un servidor y los usuarios acceden a ella desde la página web o la aplicación móvil. **y esto no se si iria aqui**

Para poder crearla **he seguido un orden** que prepare antes de nada para que en el momento de trabajar con este proyecto sea de la manera más organizada. El orden que he seguido es:

La **base de datos** fue lo primero que hice, ya que todo el proyecto depende directamente de los datos que se van a almacenar, como usuarios, tiendas o productos.

Como ya tenía la base de datos y necesitaba que estos datos se pudieran crear, consultar, modificar y eliminar la información de esta misma, entonces aplique las **operaciones CRUD**.

Además para que las contraseñas no sufran una vulnerabilidad tuve que implementar el uso de **Bcrypt** que genera y gestiona una salt aleatoria para estas contraseñas.

También hice los **formularios** para que tanto los usuarios y las tiendas puedan introducir la información dentro de la aplicación. Estos formularios envían los datos, se **envían** de manera correcta **a la base de datos** y se gestionan mediante el CRUD.

Preparé una **paleta de colores** correspondiente con lo que quería que mi aplicación transmitiese y que su elección tuviese armonía, la hice con el **creador de paletas de Material 3**.

Dejar link o imagen de la paleta de colores

Antes de comenzar a desarrollar la aplicación, hice un **mockup en Figma** con todas las pantallas para tener una idea de como quisiera que se viera la aplicación. Esto me ha permitido tener una mayor **organización de la estructura** y así poder decidir donde van a estar todos los elementos interactivos o decorativos. (podría poner una imagen aquí porque no hay abajo)



Después empecé con la **página web**, es una parte muy importante ya que desde esta se puede **acceder desde todos los dispositivos** como ordenador, móviles o tablets **sin descargar** una aplicación.

(el orden es así o al revés, mejor al revés creo)

Subí la **base de datos a la nube** para que la aplicación pueda funcionar desde cualquier lugar y no dependa únicamente de un ordenador local. De esta manera, la página web y la aplicación móvil pueden acceder a la base de datos a través de internet.

(el orden es así o al revés)

Luego creé una **API REST** para que la página web y la aplicación móvil puedan **comunicarse con la base de datos** y así los usuarios puedan enviar, recibir y actualizar todos los datos sin que los usuarios tengan acceso directo a la base de datos.

(poner algo en la línea de arriba para gastar 1 línea mas)

SEGURIDAD BASICA [https](https://) y eso

Una de las partes más importantes fue la creación de la aplicación móvil, que fue desarrollada en Android Studio (poner jetpack compose?), esta fue una de las partes más laboriosas (poner porque).

Limitaciones del Proyecto

El desarrollo de este proyecto se ha pensado desde el principio como un proyecto que **prioriza la sostenibilidad** y el beneficio comunitario sobre el beneficio económico. Por este motivo, **no se han podido implementar ciertas funcionalidades** que una aplicación como esta podría llegar a tener, esto se podría solucionar a futuro para poder ampliar el mercado de la aplicación y así llegar a más gente.

- El **usuario debe ir presencialmente a por el producto**, debido a limitaciones económicas ya que esta aplicación no genera ingresos y sería inviable.
- Esta aplicación móvil está **desarrollada** únicamente para el **sistema operativo de Android**, por lo que la gente que entre desde iOS debería hacerlo desde la página web.
- El proyecto está **desarrollado** exclusivamente **en castellano** y en el caso de querer que el proyecto se haga más grande habría que traducirlo, pero de momento está solo enfocado para España.

Funcionalidades de la aplicación

Funciones para usuarios

- Registro **HABRA QUE AÑADIR UN BUCADOR DE TIENDASS!!!!!!!!!!**
- Inicio de sesión
- Ver tiendas cercanas
- Reservar productos
- Ver categorías de tiendas
- Ver su propio perfil

Funciones para tiendas

- Crear tienda
- Subir productos
- Editar productos
- Eliminar productos

Funciones generales

- buscador
- filtros
- imágenes
- categorías

Especificaciones técnicas y seguridad

No se refieren a "qué" hace la app, sino a **cómo** es el sistema por dentro (sus cualidades). Definen restricciones, rendimiento, seguridad y experiencia de usuario.

Ejemplos aplicados a tu proyecto:

- El proyecto está alojado en la nube (AWS poner nombre) para que la base de datos sea accesible las 24 horas desde cualquier lugar.

- Las **contraseñas** de los usuarios están **cifradas** en la base de datos y no se guardan en texto plano.
- La **interfaz** de la aplicación sigue las guías de **diseño de Material 3** para ser intuitiva y moderna. **usar MATERIAL 333333333333333333**
- **RNF-4 Interoperabilidad (API REST):** La comunicación entre el frontend (web/móvil) y el backend debe realizarse mediante el intercambio de objetos JSON a través de una API REST. esto lo hace flask
- **RNF-5 Rendimiento:** El tiempo de respuesta de la API al cargar la lista de tiendas no debe superar los 2 segundos en condiciones normales de red.
- Está creado para que el servicio sea **accesible** tanto desde un **navegador web** como desde un **dispositivo Android**.

Flujo de funcionamiento de la aplicación

En esta parte vamos a ver como sería el **funcionamiento completo** del uso de la aplicación.

Lo primero que debemos hacer es **registrarnos** en el caso de que nunca se haya usado la aplicación, debemos insertar los datos que se piden y así poder registrarnos, en el caso de que ya se tenga una cuenta deberemos de **iniciar sesión**.

Una vez se inicia la sesión nos llevará a la **página principal**, los usuarios pueden ver las tiendas que **aparecen por orden de distancia**.

En el caso de que se quiera **filtrar las tiendas** por tipo de producto el usuario deberá de ir a la pestaña de categorías y seleccionar la que quiera. **(poner lo de abajo también)**

Una vez el usuario **entra a la tienda** de la cual quiere ver los productos, aparece la portada de la tienda con todos sus datos correspondientes y una imagen, debajo de estos están los productos, selecciona los que quiera y se les añade al carrito.

- **cuando ya estaba desarrollando la app pensé que un usuario podría hacer pedido y no ir a por ellos entonces puse un tiempo de 2h para ir si no se cancela el pedido**

Accede al carro para ver los productos que has añadido a él para luego darle al botón de reservar productos y estos se te reservan para que puedas ir a recogerlos a la tienda.

Ahora el usuario tiene que ir a **recoger el producto** a la tienda (y la tienda confirma que han recogido el pedido).

El usuario puede acceder a su **perfil** y desde ahí ver su nombre y toda la información que dispone además de **los datos de el peso de productos que ha salvado**.

Las **tiendas** pueden acceder a su **panel de gestión** para añadir, modificar, eliminar o confirmar productos.

(HACER DIAGRAMA DE FLUJO)

DIAGRAMA DE CASOS DE USO

Exactamente, pero con una diferencia clave: mientras que el **Flujo de funcionamiento** explica el "cómo a paso" (primero hago A, luego paso B y termino en C), el **Diagrama de Casos de Uso** se centra en los **permisos y posibilidades**.

Para que lo entiendas perfectamente, aquí tienes la comparación:

1. **Flujo de funcionamiento** (Lo que ya vimos)

Es como una **película**. Cuenta la historia en orden cronológico:

1. El usuario se registra.
2. El usuario busca una tienda.
3. El usuario reserva.
4. El usuario va a la tienda.

2. **Diagrama de Casos de Uso** (Lo que te pido el punto 2.4)

Es como el **mapa de un edificio**. No dice en qué orden entras a las salas, sino a qué salas tiene acceso cada persona:

- El Actor Usuario tiene "Acceso" para: Buscar, Reservar, Ver Perfil.
- El Actor Tienda tiene "Acceso" para: Crear Producto, Borrar Producto, Editar Tienda.
- El Actor Administrador (o cualquier) tiene "Acceso" para: Reservar productos, Ver todas las ventas.

¿Cómo dibujarlo para tu proyecto?

Si tienes que hacerlo, visualízalo así (puedes usar una herramienta como Draw.io que es gratis):

- Fuera de un cuadro (Actores): Dibuja un monigote llamado "Usuario" a la izquierda y otro llamado "Dueño de Tienda" a la derecha.
- Dentro de un cuadro (Tu App): Dibuja líneas con las acciones principales:
 - Login
 - Ver productos
 - Reservar incidente
 - Gestionar Stock
- Las flechas: ~ Tres flechas desde el Usuario a Login, Ver productos y Reservar.
 - Tres flechas desde la Tienda a Login y Gestionar Stock.

¿Por qué poner ambos en la memoria?

- El Diagrama de Casos de Uso (2.4) le dice al profesor: "¿Qué cosas hay en mi sistema y qué permisos tiene cada uno?"
- El Flujo de funcionamiento le dice al profesor: "¿Se cómo navega el usuario por mi interfaz y qué pasos sigue?"

Mi recomendación: En el apartado 2.4 por el dibujo (el diagrama de monigotes y líneas) y añado una frase diciendo: "A diferencia del Flujo de funcionamiento, este diagrama muestra las capacidades y límites de cada perfil de usuario dentro del sistema"

3

Diseño de la interfaz

Aquí están las pantallas que tiene mi aplicación.

- Pantalla de registro
- Pantalla de inicio de sesión
- Pantalla principal
- Pantalla de tienda
- Pantalla de categorías
- Perfil del usuario
- *Panel de tienda*

Diseño de la base de datos (poner el CRUD)

Para el correcto funcionamiento de la aplicación lo primero que hice fue diseñar una base de datos la cual permite **almacenar toda la información** de la app de manera organizada y coherente. La base de datos es una **de las partes más importantes** del proyecto, ya que sin ella no se podrían guardar los datos de por ejemplo los usuarios, tiendas, **imágenes** o productos que se publican en la aplicación.

La base de datos ha sido desarrollada en “como ha sido desarrollada”

Estas son las tablas que **forman la base de datos**:

las contraseñas se almacenan cifradas

USE UNIQUE (usuario_id, producto_id) PARA EVITAR DUPLICADOS

mirar esto de los indices a ver si sirve

CREATE INDEX idx_productos_tienda ON productos(tienda_id);

PONER para qué sirve cada una de ellas y cómo se relacionan entre sí mediante claves primarias y claves foráneas.



Formularios (explicar algo como se enlaza flask)

Los formularios hacen posible la **interacción entre los usuarios y el sistema**. Su función es **recoger la información** necesaria de manera estructurada para que el Backend pueda procesarla y guardarla en la base de datos.

Los formularios han sido desarrollados en “como ha sido desarrollada”

Los formularios que he usado en mi proyecto son:



mirar que esten bien todos los formularios

4

Tecnologías utilizadas

Backend

(Java / Spring Boot o lo que uses) (flask va en backend o donde va?)

Flask (Python): Se ha utilizado este micro-framework de Python para gestionar la lógica del servidor. Es el encargado de recibir las peticiones, comunicarse con la base de datos MySQL y devolver las respuestas en formato JSON.

Bcrypt: Aunque se usa dentro del código de Python, se clasifica aquí como la herramienta de seguridad para el hashing de contraseñas, garantizando que estas no se almacenen en texto plano.

Base de datos

(MySQL)

Web

HTML: Utilizado para crear la página web sin que haga falta descargar ninguna aplicación, esto hace que sea accesible desde cualquier dispositivo con acceso a páginas web.

CSS: Empleado para el diseño visual y la maquetación responsiva, asegurando que la web se adapte a diferentes tamaños de pantalla. Se ha utilizado la función `url_for` de Flask para la gestión dinámica de las hojas de estilo.

Jinja2: Motor de plantillas de Flask que permite renderizar contenido dinámico directamente en el HTML.

(JavaScript)

App móvil

Android Studio: Es el entorno de desarrollo que he utilizado para la creación de la aplicación móvil.

Jetpack Compose: Es el kit de herramientas moderno para construir la interfaz de usuario de forma declarativa. Ha permitido crear una experiencia de usuario más fluida y profesional siguiendo las guías de Material 3.

Otros

Figma: Lo he utilizado para la fase de diseño y prototipado de la aplicación móvil. Gracias a esto pude ver como seria la experiencia del usuario y el diseño visual antes de ponerme a programar.

API REST: Es la arquitectura usada para la comunicación entre el frontend de la página web y la aplicación con el backend **mediante el intercambio de objetos JSON**.

GitHub: Use github para el control de versiones de mi proyecto y por si necesitaba descargar el proyecto desde cualquier sitio. PONER FOTO O LINK

Postman: Herramienta esencial para el testeo de los *endpoints* de la API antes de integrarlos en las aplicaciones. PONER FOTO

API REST (como lo hice, que es etc)

La API REST es el "puente" que he construido con Flask para **conectar mi base de datos con la aplicación móvil y la página web.** **"como ha sido desarrollada"**

En lugar de que cada parte del proyecto funcione por separado, la **API centraliza todo**, gracias a esto he conseguido:

- **Un único punto de control:** Todo lo que pasa en la app (hacer un login, subir un producto o reservar algo) pasa por este puente.
- **Lenguaje común (JSON):** La API hace que el servidor, el móvil y la web hablen el mismo idioma. Así, no importa desde dónde entre el usuario, la información siempre se envía y recibe de la misma forma.
- **Orden en el código:** Al usar esta estructura, el diseño de la app queda totalmente separado de la programación del servidor, lo que hace que el proyecto sea mucho más limpio y profesional.

- qué endpoints tiene

Ejemplo:

- GET productos
- POST usuario
- PUT producto
- DELETE producto



Seguridad básica

Para la **gestión de las contraseñas** que han sido almacenadas en la base de datos no solo me bastaba con guardarlas ya que eso es algo muy peligroso. Para que esto no suceda he implementado el uso de **Bcrypt** para poder hacer encriptación. **"como ha sido desarrollada"**

Bcrypt genera y gestiona una **salt**, que es una cadena aleatoria que es única para cada contraseña. Esto evita que dos usuarios no puedan tener el mismo hash en la base de datos.

El funcionamiento de **Bcrypt** es el siguiente:

- Cuando un usuario o una tienda crea su cuenta, la aplicación recibe la contraseña, esta es codificada en bytes y se **genera el hash** mediante **bcrypt.hashpw()**. Esto genera la contraseña hasheada y es almacenada en la base de datos
- Cuando el usuario inicia sesión, el sistema **recupera el hash** de la base de datos y utiliza **bcrypt.checkpw()**. Esta función verifica si la contraseña introducida coincide con el has almacenado en la base de datos **y si coincide (que hace)**.



(en la imagen una contraseña encriptada)

También se ha aplicado seguridad en el proyecto mediante un sistema basado en **roles de usuario**. Esto lo añadido a la sección de seguridad ya que no todos las personas que tienen acceso a la aplicación tienen los mismos permisos y que así solo puedan acceder las personas seleccionadas.

Actualmente se han creado **tres** tipos de perfiles:

- **Usuario (Cliente):** Este perfil permite navegar por la aplicación, filtrar por categorías, ver tiendas, productos y reservarlos. Estos **no tienen permisos**

para poder acceder a funciones de inventario ni paneles de gestión de tiendas.

- **Tienda:** El perfil de las tiendas cuenta con varios permisos que los clientes no. Estos pueden crear, editar y eliminar productos, además de confirmar la recogida de los pedidos. El **acceso** de estas tiendas está **limitado** a su propio establecimiento, no pueden modificar los datos de otras tiendas.(aplicar)



- **Administradores:** Los encargados de supervisar el correcto funcionamiento del proyecto y asegurar que el contenido subido a las tiendas cumplen con las normas de la aplicación, **tienen acceso a todas las funcionalidades** de la aplicación.

- **conexión HTTPS** **HACER**

como hacer que no me inyecten comandos SQL en ningun lado

(poner imagen)

Página web (como lo hice, que es etc, que sea HTTPS)

La página web es una de las partes más importantes del proyecto, ya que esta **permite** que cualquier persona pueda **acceder a ella desde varios dispositivos diferentes** y **sin tener la necesidad de descargar** ninguna aplicación. La diseñé de tal manera que fuera sencilla y funcional. **Cómo está conectada a la misma API que la aplicación móvil esto hace que la información que se ve en la web se actualice también en la aplicación móvil.** **"como ha sido desarrollada HTML CSS"**



LA RUTA DEL CSS DEBE SER PARA QUE NO PETE CREO

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
```

Aplicación móvil (como lo hice, que es etc)

La aplicación móvil es la manera para que los usuarios y las tiendas puedan acceder a las funcionalidades del proyecto de la **manera más rápida posible**. Su diseño es sencillo y ágil, adaptado a todo tipo de públicos. Como la aplicación está desarrollada para Android esto hace que se puedan aprovechar todas las **funciones táctiles** que estas nos ofrecen. Al igual que la página web, la aplicación está conectada directamente con la API **"como ha sido desarrollada"**

.

(mas texto)



Funcionalidades multimedia

Ahora vamos a ver cómo se utilizan los **elementos visuales** para mejorar la experiencia de los usuarios, haciendo que la aplicación sea más **intuitiva y bonita**.

- Las **tiendas** deben de subir **fotografías** de sus **productos** y de su **establecimiento**, lo que hace que los usuarios tengan una idea de que tipos de productos van a recibir.
- Los **usuarios** pueden ponerse una **foto de perfil personalizada** para personalizar su perfil de la manera que ellos quieran.
- **Mapas interactivos:** Ayudan al usuario a localizar visualmente los comercios cercanos, haciendo que sea mucho más sencillo planificar la ruta para ir a por el pedido.
- **Gráficas de impacto:** Se incluyen elementos visuales para mostrar las estadísticas de ahorro y productos salvados, transformando los datos numéricos en información fácil de entender.
- **Optimización de carga:** Todo el contenido multimedia está gestionado para que la aplicación funcione con fluidez, evitando esperas innecesarias al cargar las fotos o los mapas.

Esto es para que ponga como funciona la relación de los archivos multimedia



Puedes poner:

- imágenes de productos
- imágenes de tiendas
- mapa interactivo
- gráficos
- exportación a JSON

Nube AWS

como lo he hecho y poner lo del flask

Leamer Lab: Es un curso en el que podrás acceder a la consola de AWS y poder usarla como laboratorio. En la consola podrás crear instancias de forma que estén totalmente accesibles a ellas. La idea es que para el proyecto intermodular, de acuerdo para cumplir con los RA de la asignatura, subas la BD a un entorno cloud. Como hemos estado trabajando laboratorios en AWS y además realizado una práctica sobre BD, sería ideal hacerlo en este entorno. Sin embargo, existe la posibilidad que utilices otras herramientas. Tened en cuenta que en este laboratorio hay un máximo de créditos disponibles que consumen las instancias.

Cloud Developing: En el sentido de que sería futuro desarrolladores de software, no habilito este curso para que podáis ver más en profundidad este aspecto, aunque sea en el entorno AWS. Nota: Este curso está en inglés, no hablo la opción de castellano, aunque como informático, eso no os debe resultar un impedimento :)

Cloud Operations: Aunque estaría más enfocado a un perfil de administración de sistemas, es el curso "base" para ello. Se pueden ver cosas como instalación, monitorización y solución de problemas, además varios otros servicios que ofrece AWS además de los vistos en el curso de Fundamentals.

Todos los cursos finalizan el 31 de Julio (aunque como hemos visto puede que tengáis acceso tiempo después)



Informes y gráficas

En esta sección se analiza la actividad de la aplicación mediante datos estadísticos que permiten medir el éxito del proyecto. El objetivo es transformar los números almacenados en la base de datos en información visual que sea fácil de interpretar tanto para los comercios como para los usuarios:

- Estadísticas de ahorro: Se muestran gráficas sobre el peso total de productos que han sido recuperados, permitiendo ver el impacto real en la reducción de residuos.
- Actividad por tienda: Informes detallados sobre qué establecimientos son los más activos y qué tipo de productos son los que más se salvan.
- Crecimiento de usuarios: Una comparativa visual de los nuevos registros mensuales para entender cómo está evolucionando la comunidad de la plataforma.
- Exportación de datos: La posibilidad de generar estos informes en formatos estándar como JSON para que la información pueda ser analizada externamente.

Para que la aplicación no sea solo una lista de productos y nombres, he incluido un apartado visual que resume todo lo que está pasando en **LocalHero**. En lugar de mirar tablas de datos difíciles de entender, tanto los usuarios como las tiendas pueden ver de forma clara el resultado de sus acciones:

Aquí explicas:

- usuarios registrados por mes
- productos salvados
- peso total recuperado
- estadísticas por tienda

(poner imagen)

Impacto social y ambiental

El objetivo principal del proyecto es **reducir** en mayor medida el **desperdicio de las tiendas** y que haya un **consumo más responsable**. En la actualidad, muchos productos son desechados porque su fecha de caducidad es próxima o porque no han sido vendidos en la temporada que debería de haber sido vendida. Esto puede ocurrir tanto como en supermercados, tiendas de ropa, jardinería, ultramarinos, hornos y muchos otros establecimientos.

En esta aplicación también se nos permite que los usuarios puedan ver la **cantidad de productos y el peso estimado** que han sido **salvados** y así **fomentar más el consumo sostenible**.

Desde el punto de vista **social**, mi proyecto puede **ayudar a personas que necesitan productos** y que a lo mejor no se lo pueden permitir o tienen que gastar el dinero en otras cosas de primera necesidad. Gracias a esta aplicación, esta gente puede acceder a productos en buen estado que podrían haber sido desechados.

AÑADIR ESTO EN ALGUN PUNTO DE SILVIAAAAAAAAAAAAAA y de ra 1 y 2

6. Gestión de Stock en Tiempo Real "Crítico"

"El sistema gestiona el stock de forma asíncrona. Si dos usuarios intentan reservar el último producto exactamente en el mismo milisegundo, el sistema podría presentar una condición de carrera (*race condition*). Al ser un prototipo de impacto social, no se han implementado bloqueos de base de datos a nivel de fila (*row-level locking*) extremadamente complejos que serían necesarios en una plataforma de ventas masivas."

Problemas encontrados durante el desarrollo

A lo largo del desarrollo de mi proyecto he ido encontrando varios **problemas** los cuales me han hecho que no vaya tan rapido, pero **me han hecho aprender** y así en un futuro ya podré saber como solucionar estos problemas. Estos han sido algunos de los problemas que me han ido surgiendo:

Ejemplo:

- errores en la base de datos
- problemas con la API
- errores en el login
- problemas al subir imágenes
- cuando ya estaba desarrollando la app pensé que un usuario podría hacer pedido y no ir a por ellos entonces puse un tiempo de 2h para ir si no se cancela el pedido

Mejoras futuras

Al tener tiempo limitado para la entrega de este proyecto, no se han podido implementar todas las funciones que me gustaría porque para implementarlas necesitaría mucho más tiempo, unas de ellas serían:

- Añadir la función de poder **poner establecimientos en favoritos**, y que estos cuando suban un producto nuevo a su tienda que al usuario le llegue una **notificación** para que sea el primero en enterarse.
- Para fomentar el uso de la app, se podría añadir un **sistema de logros y niveles**. Los usuarios ganarían medallas virtuales y podrían compartir sus estadísticas en **redes sociales** y así se le daría más visualización al proyecto.

- Implementar la aplicación también para **dispositivos iOS** y así ampliar más nuestra aplicación ya que ahora solo pueden acceder a través de la **página web**.
- El uso de **algoritmos de Machine Learning** para analizar los pedidos del usuario. La aplicación podría aprender qué tipos de productos suele recoger y enviarle **recomendaciones personalizadas**.
- Poner la pagina en **más idiomas** para poder abrir la aplicación a mas paises y asi poder ayudar al mundo a reducir sus desechos.

Conclusión final

Hacer este proyecto ha sido tanto **desafiante** como **satisfactorio** ya que de una idea que hemos tenido a principio de curso y **he podido aprender** mucho de **cómo se puede llegar a realizar** los proyectos y he **tocado todas las partes** para que este esté completo y funcional, empezando desde una base de datos y acabando desde una aplicación móvil.

Como ya he hecho el desarrollo de un proyecto desde 0, el siguiente que haga ya saldría **más fluido** ya que tengo una base de cómo desarrollarlo y sería todo mucho más rápido y me podría centrar en cosas más específicas para dar un producto mucho más pulido y con **funciones nuevas** que no se hayan implementado en este y así **aprender cosas nuevas**.

“añadir despedida y gracias a todos los profesores por enseñarme todo”

Gracias.

Aquí explicas:

- qué has aprendido
- si te ha gustado el proyecto
- si te ha resultado difícil
- si crees que podrías mejorarlo